SPRINT 0

Preparación Técnica y Logística del Proyecto

FinanzasU — Plataforma Web de Gestión Financiera Personal

Campo	Detalle
Proyecto	FinanzasU — Plataforma Web de Gestión Financiera
Contrato	FINU-2026-001
Cliente (PO)	FinanzasU S.A.S. — Juan Camilo Triana
Desarrollador	Rise — Deibyd Castillo
Tipo de fase	Sprint 0 — Preparación (no es un sprint de entrega de funcionalidades)
Duración estimada	Días previos al Sprint 1 — máximo 1 semana
Objetivo	Sentar las bases técnicas, organizativas y de acuerdo para que el Sprint 1 comience sin interrupciones
Supervisor académico	Gustavo Adolfo Osorio — Universidad del Valle, Sede Zarzal · 2026

¿Qué es el Sprint 0?

En el marco de las metodologías ágiles (Scrum), el Sprint 0 no es un ciclo de desarrollo propiamente dicho, sino una fase de preparación técnica y logística que ocurre antes de que comience el primer sprint de construcción del producto.

> **Objetivo del Sprint 0:**
> Su propósito no es entregar funcionalidades al cliente, sino resolver la 'deuda técnica' antes de que exista: configurar entornos, acordar estándares, preparar el repositorio y alinear al equipo para que el Sprint 1 comience sin fricciones ni sorpresas.

A favor del Sprint 0	En contra del Sprint 0
Evita que el Sprint 1 sea un caos de instalaciones y configuraciones	Puede convertirse en una fase de diseño extenso (estilo cascada) que retrasa la entrega de valor real
Alinea al equipo técnico y al cliente sobre expectativas, estándares y herramientas	En proyectos pequeños puede resultar excesivo si el equipo ya tiene experiencia previa juntos
Define la Definición de Terminado (DoD) antes de escribir la primera línea de código	La guía oficial de Scrum no reconoce el Sprint 0 — es una práctica informal ampliamente adoptada

1. Definición de la Visión del Producto

1.1. Visión del negocio

> **Visión:**
> Construir la plataforma web de gestión financiera personal más accesible y usable para estudiantes universitarios colombianos; una herramienta que elimine la barrera de entrada a la educación financiera personal mediante una experiencia simple, visual y sin vinculación bancaria.

1.2. Alineación del equipo — Preguntas clave respondidas

Pregunta	Respuesta acordada
¿Qué problema resuelve?	Los estudiantes universitarios no tienen herramientas simples y adaptadas a su contexto para gestionar sus finanzas personales
¿Quién es el usuario principal?	Estudiantes universitarios de 17 a 28 años, sin experiencia financiera previa, en Colombia
¿Cuál es el MVP?	Plataforma web con autenticación, CRUD de transacciones, presupuesto mensual con alertas y dashboard con gráficas Recharts
¿Qué NO incluye el MVP?	Sin vinculación bancaria, sin notificaciones push, sin app móvil nativa, sin roles de administrador, sin modo offline
¿Cuál es el criterio de éxito del proyecto?	Las 26 historias terminadas, 8+ testers satisfechos (≥3.5/5) y documentación completa entregada en 75 días
¿Quién aprueba cada sprint?	EL CLIENTE (Juan Camilo Triana) firma el Acta de Aceptación dentro de los 3 días hábiles tras cada Sprint Review

2. Stack Tecnológico Confirmado

El siguiente stack fue acordado entre EL CLIENTE (FinanzasU S.A.S.) y EL DESARROLLADOR (Rise) y queda formalizado como parte del Contrato N° FINU-2026-001 y el Anexo 2 (Especificaciones Técnicas).

Capa	Tecnología	Versión	Decisión de uso
Frontend	React	18+	Framework de UI con componentes funcionales y hooks
Bundler	Vite	5+	HMR instantáneo y build optimizado; reemplaza Create React App
Estilos	Tailwind CSS	3+	Utility-first — responsive sin escribir CSS personalizado
Gráficas	Recharts	2+	Nativa para React, simple de integrar con hooks y datos de Supabase

Backend & DB	Supabase	Latest	PostgreSQL + Auth + RLS + Edge Functions — sin servidor propio
Autenticación	Supabase Auth	Latest	JWT + bcrypt — sin sesiones de servidor, seguro y escalable
Seguridad de datos	Row Level Security (RLS)	—	Cada usuario solo accede a sus propios datos en PostgreSQL
Control versiones	Git + GitHub	—	Flujo main → develop → feature/HU-XX con PR obligatorio

2.1. Arquitectura definida

Decisión arquitectónica	Valor elegido	Alternativa descartada	Razón
Tipo de arquitectura	SPA (Single Page Application) con React	MPA con PHP / servidor tradicional	Mejor UX, sin recargas de página, ideal para dashboards
Backend	Serverless con Supabase	Backend propio con Node.js + Express	Reduce tiempo de desarrollo; Auth, DB y API en un solo servicio
Base de datos	PostgreSQL (Supabase)	MySQL / MongoDB	Relacional, soporte nativo de RLS, integrado con Supabase Auth
Estilo de API	Supabase JS Client + RPC Functions	API REST propia / GraphQL	API autogenerada por Supabase; sin endpoints manuales que mantener
Estrategia de estilos	Tailwind CSS utility-first	CSS puro / Bootstrap / Styled Components	Velocidad de desarrollo, consistencia visual, responsive nativo
Convención de nombres	camelCase en JS/JSX, snake_case en SQL/Supabase	—	Convención estándar de cada ecosistema

3. Configuración del Entorno de Desarrollo

3.1. Repositorio GitHub

#	Ítem	Estado
1	Crear el repositorio finanzasU en GitHub con visibilidad privada o pública según acuerdo	Pendiente
2	Inicializar el proyecto React con Vite: npm create vite@latest finanzasU -- --template react	Pendiente
3	Crear las ramas base: main (producción) y develop (integración del sprint)	Pendiente
4	Configurar la rama main como protegida: requerir PR y al menos 1 aprobación antes de fusionar	Pendiente
5	Crear el archivo .gitignore con: node_modules/, .env.local, dist/, .DS_Store, Thumbs.db	Pendiente
6	Invitar a todos los integrantes del equipo Rise como colaboradores del repositorio	Pendiente
7	Configurar GitHub Projects con el tablero Kanban: Backlog → Sprint Actual → En progreso → En revisión → Terminado	Pendiente

3.2. Estrategia de ramas (Git Flow simplificado)

```
main          ← producción — solo recibe merges desde develop con PR aprobado
└── develop   ← integración del sprint — base para todas las features
    ├── feature/HU-01-esquema-supabase   ← rama por historia
    ├── feature/HU-03-auth-supabase
    ├── feature/HU-05-login-jsx
    └── fix/HU-09-filtro-transacciones   ← correcciones
```

3.3. Configuración de Supabase

#	Ítem	Estado
1	Crear el proyecto en Supabase: supabase.com → New Project → nombre: finanzas-u	Pendiente
2	Ir a Settings → API y anotar: Project URL y anon public key	Pendiente
3	Habilitar Row Level Security (RLS) en todas las tablas desde el inicio	Pendiente
4	Crear las tablas ejecutando el script supabase/migrations/001_initial_schema.sql en el SQL Editor	Pendiente
5	Aplicar las políticas RLS ejecutando supabase/policies.sql	Pendiente
6	Insertar los datos de prueba con supabase/seed.sql	Pendiente
7	Verificar en Supabase Studio que las tablas y políticas se crearon correctamente	Pendiente

8	Habilitar Supabase Auth con proveedor Email/Password en Authentication → Providers	Pendiente

3.4. Variables de entorno

Crear el archivo .env.example en la raíz del proyecto con las siguientes variables (sin valores reales):

```
# .env.example — copiar a .env.local y completar con los valores reales del proyecto Supabase
# NUNCA subir .env.local al repositorio Git
VITE_SUPABASE_URL=https://xxxxxxxxxxxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

#	Ítem	Estado
1	Crear .env.example en la raíz del proyecto con las variables documentadas arriba	Pendiente
2	Agregar .env.local al .gitignore (verificar que ya esté incluido)	Pendiente
3	Compartir los valores reales del .env.local con el equipo por canal privado (no por GitHub)	Pendiente
4	Crear el archivo src/services/supabaseClient.js con la inicialización del cliente Supabase	Pendiente

4. Arquitectura Inicial del Proyecto

4.1. Estructura de carpetas a crear antes del Sprint 1

```
finanzasU/ ├── src/ │ ├── components/ │ │ ├── layout/ # Navbar.jsx, Layout.jsx, ProtectedRoute.jsx │ │ ├── ui/ # Modal.jsx, ProgressBar.jsx, Spinner.jsx, AlertBadge.jsx │ │ └── charts/ # PieChart.jsx, BarChart.jsx, LineChart.jsx │ ├── pages/ # Login.jsx, Register.jsx, Dashboard.jsx, │ │ # Transacciones.jsx, Categorias.jsx, Presupuestos.jsx, Perfil.jsx │ ├── hooks/ # useAuth.js, useTransacciones.js, useCategorias.js, │ │ # usePresupuestos.js, useDashboard.js │ ├── services/ # supabaseClient.js, transaccionesService.js, │ │ # categoriasService.js, presupuestosService.js, dashboardService.js │ ├── utils/ # formatMoneda.js, exportCSV.js, constants.js │ ├── App.jsx # Router principal con rutas protegidas │ └── main.jsx # Punto de entrada ├── supabase/ │ ├── migrations/001_initial_schema.sql │ ├── policies.sql │ └── seed.sql ├── docs/ # diagrama-ER.png, API.md, rls-policies.md ├── .env.example ├── .gitignore ├── vite.config.js ├── tailwind.config.js └── package.json
```

#	Ítem	Estado
1	Crear la estructura de carpetas src/components, src/pages, src/hooks, src/services, src/utils	Pendiente
2	Crear la carpeta supabase/ con los archivos SQL: migrations/, policies.sql y seed.sql	Pendiente

3	Crear la carpeta docs/ con espacio para diagrama-ER.png, API.md y rls-policies.md	**Pendiente**
4	Crear archivos placeholder vacíos (.gitkeep) en carpetas vacías para que Git las versione	**Pendiente**
5	Crear App.jsx con el Router de React Router DOM y las rutas protegidas con ProtectedRoute.jsx	**Pendiente**

4.2. Modelo de datos inicial — Diagrama ER

El diagrama ER y el script SQL inicial deben estar listos antes de comenzar el Sprint 1 para que todos los módulos trabajen sobre el mismo esquema de base de datos.

Tabla	Columnas principales	Relaciones
categorias	id (UUID PK), nombre, tipo, usuario_id, icono, es_predeterminada	usuario_id → auth.users (NULL para predeterminadas)
transacciones	id (UUID PK), usuario_id, categoria_id, monto, tipo, descripcion, fecha	usuario_id → auth.users | categoria_id → categorias
presupuestos	id (UUID PK), usuario_id, categoria_id, monto_limite, mes, anio	usuario_id → auth.users | categoria_id → categorias | UNIQUE(usuario, categoria, mes, anio)

5. Estándares de Calidad y Código

5.1. Linter y formateo

#	Ítem	Estado
1	Instalar y configurar ESLint: npm install --save-dev eslint eslint-plugin-react	Pendiente
2	Instalar y configurar Prettier: npm install --save-dev prettier eslint-config-prettier	Pendiente
3	Crear el archivo .eslintrc.json con reglas básicas para React y hooks	Pendiente
4	Crear el archivo .prettierrc con: singleQuote: true, semi: false, tabWidth: 2	Pendiente
5	Configurar el script en package.json: "lint": "eslint src --ext .js,.jsx"	Pendiente
6	Verificar que todos los integrantes tienen la extensión Prettier instalada en VS Code	Pendiente

5.2. Convenciones de código acordadas

Elemento	Convención	Ejemplo
Componentes React	PascalCase	TransaccionCard.jsx, PresupuestoItem.jsx
Hooks personalizados	camelCase con prefijo use	useTransacciones.js, useDashboard.js
Services	camelCase con sufijo Service	transaccionesService.js
Variables y funciones	camelCase	totalIngresos, handleSubmit, formatMoneda
Columnas SQL	snake_case	usuario_id, monto_limite, es_predeterminada
Ramas Git	kebab-case con prefijo	feature/HU-05-login-jsx, fix/HU-12-filtro
Commits	Conventional Commits	feat: agregar formulario de transacción
Archivos CSS/Tailwind	clases de Tailwind directamente	className="flex items-center gap-4"

5.3. Estrategia de testing

Para el contexto académico del proyecto, el testing será principalmente manual. Se establece la siguiente estrategia mínima:

Tipo de prueba	Herramienta	Cuándo se ejecuta	Responsable

Pruebas funcionales manuales	Guía de casos de prueba del Sprint 5	Sprint 5 — con 8+ testers reales	Rise + FinanzasU S.A.S.
Pruebas de regresión básica	Manual — recorrer flujos principales	Al terminar cada historia (DoD)	Dev asignado a la historia
Pruebas de endpoints Supabase	Postman	Al terminar cada servicio o RPC	Dev que implementó el service
Pruebas de UI responsive	DevTools del navegador (Device Mode)	Al terminar cada página React	Dev 1 — Frontend Lead
Pruebas de seguridad RLS	Supabase Studio + Postman con 2 cuentas	Al configurar cada política RLS	Dev encargado de Supabase

6. UI/UX y Requerimientos de Diseño

6.1. Handoff de diseño — Figma

#	Ítem	Estado
1	Crear el proyecto en Figma: 'FinanzasU — Diseño UI'	**Pendiente**
2	Diseñar los mockups de las 7 páginas principales: Login, Register, Dashboard, Transacciones, Categorías, Presupuestos, Perfil	**Pendiente**
3	Definir la paleta de colores del proyecto: color primario, secundario, de error, de éxito y neutros	**Pendiente**
4	Definir la tipografía: familia, tamaños (h1–h3, body, caption) y pesos	**Pendiente**
5	Diseñar los componentes reutilizables: botones, inputs, tarjetas, badges, barras de progreso y modales	**Pendiente**
6	Compartir el enlace de Figma con todos los devs del equipo Rise antes del Sprint 1	**Pendiente**

6.2. Principios de diseño acordados

Principio	Aplicación en FinanzasU
Simple antes que completo	Formularios con el mínimo de campos obligatorios; opcionales claramente marcados
Colores con significado	Verde = ingreso / positivo, Rojo = gasto / alerta crítica, Amarillo = advertencia
Feedback inmediato	Spinner en todas las operaciones > 500ms; confirmación verde tras guardar exitosamente

Mobile-first	Diseñar primero para 375px y escalar hacia arriba; menú hamburguesa en < 768px
Mensajes en español claro	Sin mensajes técnicos ni en inglés visibles al usuario final
Estados vacíos con orientación	Cuando no hay datos, mostrar ícono + mensaje + botón de acción ('Registra tu primera transacción')

7. Refinamiento del Backlog y Definición de Terminado

7.1. Product Backlog — Resumen por sprint

El backlog completo de 26 historias de usuario está documentado en el documento 'Backlog_Cards_FinanzasU.docx'. A continuación se presenta el resumen por sprint para alinear al equipo antes del Sprint 1:

Sprint	Período	Historias	Módulos clave	Responsables
Sprint 1	Días 1–15	HU-01 a HU-08 (8)	Supabase Auth, esquema DB, Login/Register, Layout, Perfil	Todos los devs de Rise
Sprint 2	Días 16–30	HU-09 a HU-12 (4)	CRUD transacciones, filtros, paginación, exportación CSV	Todos los devs de Rise
Sprint 3	Días 31–45	HU-13 a HU-18 (6)	Categorías personalizadas, presupuestos, alertas 80%/100%	Todos los devs de Rise
Sprint 4	Días 46–60	HU-19 a HU-22 (4)	Dashboard, gráficas Recharts (torta, barras, línea, top 5)	Todos los devs de Rise
Sprint 5	Días 61–75	HU-23 a HU-26 (4)	Seguridad RLS, pruebas reales, corrección bugs, docs	Todos los devs de Rise

7.2. Historias del Sprint 1 — verificación de criterios de aceptación

Las siguientes historias deben tener criterios de aceptación claros y aprobados por EL CLIENTE antes de comenzar el Sprint 1:

ID	Título	Criterios de aceptación listos	Dev asignado
HU-01	Esquema e inicialización de la base de datos	✓ **Listos**	Dev 4
HU-02	Modelo de Usuario y conexión Supabase	✓ **Listos**	Dev 4
HU-03	Controlador de registro, login y sesión	✓ **Listos**	Dev 3 (Supabase)
HU-04	Servicio de perfil y cambio de contraseña	✓ **Listos**	Dev 3 (Supabase)

HU-05	Diseño React/Tailwind de páginas de autenticación	✓ **Listos**	Dev 1
HU-06	Layout general, Navbar y página de perfil	✓ **Listos**	Dev 1
HU-07	Validaciones JS y llamadas a Supabase Auth	✓ **Listos**	Dev 2
HU-08	Toggle navbar responsive y lógica de perfil	✓ **Listos**	Dev 2

7.3. Definición de Terminado (DoD) — Acuerdo formal del equipo

Una historia de usuario se considera TERMINADA únicamente cuando cumple la totalidad de los siguientes criterios, acordados por Rise y FinanzasU S.A.S. antes del inicio del Sprint 1:

1	El código está subido a GitHub en la rama feature/HU-XX con al menos un commit descriptivo (feat:, fix:, etc.)
2	Se realizó un Pull Request hacia develop y fue revisado y aprobado por al menos un integrante del equipo Rise
3	La funcionalidad fue probada manualmente en el escenario exitoso y en al menos un escenario de error antes del PR
4	Si la historia involucra datos de Supabase, las políticas RLS fueron verificadas con 2 cuentas distintas
5	La interfaz es funcional y visualmente correcta en resoluciones de 375px (móvil) y 1280px (escritorio)
6	Todos los mensajes visibles al usuario están en español claro, sin términos técnicos ni mensajes de error de Supabase expuestos
7	Si se agregaron nuevas consultas a Supabase, están documentadas o al menos comentadas en el service correspondiente
8	Ninguna funcionalidad previamente terminada fue rota por los cambios de esta historia (prueba de regresión básica)

8. Herramientas de Comunicación y Gestión

Herramienta	Uso en el proyecto	Responsable de configurar
GitHub Projects	Tablero Kanban del backlog con las 26 historias organizadas por sprint	Deibyd Castillo (Rise)
GitHub Issues	Cada historia de usuario es una Issue con su label, assignee, points y milestone	Deibyd Castillo (Rise)
WhatsApp / Discord	Comunicación diaria del equipo + Daily Standup asíncrono si no se reúnen presencialmente	Equipo Rise
Google Meet / Zoom	Sprint Planning, Sprint Review y Sprint Retrospective	Deibyd Castillo + Juan Camilo Triana
Figma	Diseño de mockups y handoff visual al equipo de desarrollo	Dev 1 (Frontend Lead)
Supabase Studio	Gestión de la base de datos, políticas RLS, logs y autenticación	Dev encargado de Supabase
Postman	Pruebas de servicios Supabase y funciones RPC del dashboard	Dev que implementa cada service
Google Forms	Encuesta de satisfacción para los testers del Sprint 5	Dev 2 de Rise

8.1. Calendario de ceremonias Scrum

Ceremonia	Cuándo	Duración	Participantes	Entregable
Sprint Planning	Día 1 de cada sprint	~1 hora	Todo el equipo Rise + Juan Camilo Triana	Sprint Backlog acordado con historias asignadas
Daily Standup	Cada día hábil	10 min	Equipo Rise	¿Qué hice? ¿Qué haré? ¿Algún bloqueo?
Sprint Review	Día 15 de cada sprint	~30 min	Equipo Rise + Juan Camilo Triana	Demo de funcionalidades + Acta de Aceptación firmada
Sprint Retrospective	Tras el Review (mismo día)	~20 min	Equipo Rise	Lista de mejoras para el siguiente sprint

9. Checklist General del Sprint 0

El Sprint 0 se considera completado cuando todos los ítems de la siguiente lista están en estado ✅ Listo. Ningún ítem puede quedar pendiente al iniciar el Sprint 1.

V1	✅Visión del producto alineada entre Rise y FinanzasU S.A.S.	*Visión*
V2	✅Preguntas clave del producto respondidas y documentadas	*Visión*
S1	✅Stack tecnológico confirmado: React + Supabase + Tailwind + Recharts + Git	*Stack*
S2	✅Arquitectura definida: SPA + Serverless + PostgreSQL + RLS	*Stack*
R1	☐ Repositorio creado en GitHub con ramas main y develop	*Repositorio*
R2	☐ Rama main protegida con regla de PR obligatorio	*Repositorio*
R3	☐ Equipo Rise invitado como colaboradores	*Repositorio*
R4	☐ GitHub Projects configurado con tablero Kanban	*Repositorio*
R5	☐ .gitignore creado con .env.local, node_modules/, dist/	*Repositorio*
SB1	☐ Proyecto Supabase creado y credenciales compartidas con el equipo	*Supabase*
SB2	☐ Tablas creadas y políticas RLS aplicadas desde schema.sql y policies.sql	*Supabase*
SB3	☐ Supabase Auth habilitado con proveedor Email/Password	*Supabase*
SB4	☐ seed.sql ejecutado — datos de prueba disponibles	*Supabase*
E1	☐ .env.example creado con las variables documentadas	*Entorno*
E2	☐ .env.local configurado localmente en cada máquina del equipo	*Entorno*
E3	☐ supabaseClient.js creado e inicializado en src/services/	*Entorno*
P1	☐ Proyecto React creado con Vite y dependencias instaladas (npm install)	*Proyecto React*
P2	☐ Tailwind CSS configurado: tailwind.config.js y directivas en index.css	*Proyecto React*
P3	☐ Recharts instalado: npm install recharts	*Proyecto React*
P4	☐ React Router DOM instalado: npm install react-router-dom	*Proyecto React*
P5	☐ Estructura de carpetas src/ creada: components, pages, hooks, services, utils	*Proyecto React*
P6	☐ App.jsx con Router y rutas protegidas con ProtectedRoute.jsx configurado	*Proyecto React*

C1	☐ ESLint y Prettier instalados y configurados	*Código*
C2	☐ Convenciones de código documentadas y compartidas con el equipo	*Código*
D1	☐ Mockups de las 7 páginas en Figma completados y compartidos	*Diseño*
D2	☐ Paleta de colores y tipografía definidas	*Diseño*
B1	✅ 26 historias de usuario documentadas con criterios de aceptación	*Backlog*
B2	☐ Historias del Sprint 1 creadas como Issues en GitHub con labels y assignees	*Backlog*
B3	✅ Definición de Terminado (DoD) acordada formalmente por ambas partes	*Backlog*
DOC1	✅ README.md con instrucciones de instalación	*Documentación*
DOC2	✅ Especificaciones Técnicas (Anexo 2) firmadas	*Documentación*
DOC3	✅ Contrato FINU-2026-001 firmado por ambas partes	*Documentación*

> **Nota importante:**
> Los ítems marcados con ✅ ya están resueltos gracias a la documentación previa del proyecto (contrato, backlog, especificaciones técnicas, README y DoD). Los ítems con ☐ son las acciones técnicas que el equipo Rise debe completar antes de iniciar el Sprint 1.

10. Aprobación del Sprint 0

Las partes confirman que los acuerdos, estándares y configuraciones definidos en este documento son correctos y suficientes para iniciar el Sprint 1 del proyecto FinanzasU.

EL CLIENTE — FinanzasU S.A.S.	EL DESARROLLADOR — Rise
_____ **Juan Camilo Triana** Representante Legal — FinanzasU S.A.S. C.C. N° _____ *Fecha: ___/___/2026*	_____ **Deibyd Castillo** Representante Legal — Rise C.C. N° _____ *Fecha: ___/___/2026*